LOGSCOPE

A Lightweight Log Analysis and Error Diagnosis Platform

PROJECT WRITE-UP

P. GANESH PRAKADESH

23MIC0020

1. Title

LogScope: A Lightweight Log Analysis and Error Diagnosis Platform

2. Motivation

Modern software applications continuously generate log files that record events, warnings, exceptions, database activity, and system failures. When an application encounters a problem, these files can contain hundreds or thousands of lines. Developers and support teams may need to inspect them manually to identify the important error, understand its context, and decide what to investigate next.

This process can be time-consuming, particularly when several errors appear together or when one underlying issue produces multiple symptoms. Repeated investigation of familiar problems can also consume developer time that could otherwise be spent on development and improvement.

LogScope was motivated by the need for a straightforward tool that turns raw log data into organized diagnostic information. It aims to help users identify errors, understand likely causes, review possible fixes, and reuse useful solutions while keeping uploaded log content separate from persistent community knowledge.

3. Existing Gap / Event That Motivated the Project / USP

Log monitoring and analysis platforms such as Splunk and the ELK Stack provide capabilities for collecting, searching, monitoring, and visualizing logs. These are established solutions, but their setup, infrastructure requirements, configuration, cost, or learning curve may not be suitable for every individual developer or small team.

Locating an error and understanding it are different tasks. Searching for a line containing “ERROR” does not necessarily explain whether the issue is a missing database table, an unknown column, a connection limit, a programming exception, or a network timeout.

LogScope focuses on a lightweight upload-and-analysis workflow. It parses logs, identifies and groups errors, examines potentially related events, and provides rule-based explanations and troubleshooting suggestions through a dashboard.

Its intended USP is the combination of multi-category log analysis, readable diagnostic results, and a community knowledge mechanism. Users can contribute fixes anonymously, and other users can upvote individual fixes they find useful. Community submissions are kept distinct from the application's normal rule-based diagnosis. Only reviewed and appropriately sanitized information should be promoted into trusted reusable knowledge.

LogScope is intended to complement existing monitoring platforms for focused diagnostic tasks, rather than replace enterprise observability systems.

4. Architecture

LogScope follows a layered web application architecture. The frontend handles user interaction, the Flask backend receives requests and coordinates processing, the analysis modules interpret uploaded logs, and PostgreSQL stores persistent reusable knowledge.

4.1 Architecture flow

1. User selects and uploads a log file through the frontend.
2. Flask receives the upload, validates the request, and invokes the analysis components.
3. The parser reads the log and extracts available details such as timestamps, severity levels, messages, exceptions, and stack-trace elements.
4. Error detection identifies and classifies relevant patterns.
5. The analysis layer groups repeated patterns, calculates statistics, and examines whether events may be related.
6. Deterministic rules and relevant stored knowledge are used to produce possible explanations and troubleshooting suggestions.
7. Neon-hosted PostgreSQL stores reusable, generalized knowledge and related metadata. Raw uploaded logs are intended to remain temporary.
8. The backend returns structured results to the frontend dashboard.

A correlation between log events indicates a possible relationship; it does not automatically prove the root cause. Diagnosis should be treated as assistance for investigation, and users should validate fixes before applying them.

4.2 Main application components

Component	Responsibility
app.py	Flask application entry point; handles HTTP requests and coordinates operations.
logparser/log_parser.py	Parses log entries and extracts available structured information.
logparser/error_detector.py	Detects error patterns in parsed log content.
analyzer/analyzer.py	Coordinates analysis, grouping, correlation, and statistics.
analyzer/severity.py	Handles severity classification.

analyzer/ai_resolver.py	Deterministic diagnosis logic despite its historical filename; the core diagnosis does not depend on a generative AI API.
analyzer/lookup.py	Retrieves relevant diagnostic knowledge.
analyzer/knowledge.py	Knowledge-related operations, subject to the implemented module behavior.
database/db.py	Handles database connectivity and persistence operations.
templates/index.html	Defines the webpage structure.
static/app.js	Handles browser interactions and API communication.
static/style.css	Defines website styling.

5. Complete Tech Stack Information

HTML: Defines the structure and content of the web interface.

CSS: Controls layout, typography, colors, spacing, and dashboard presentation.

JavaScript: Handles frontend interactions, API requests, and rendering of returned results.

Python: Implements log parsing, text processing, pattern matching, classification, analysis, and diagnostic rules.

Flask: A lightweight Python web framework used for routes, API endpoints, file uploads, request validation, and coordination of backend modules.

PostgreSQL: A relational database used to persist structured reusable diagnostic knowledge and metadata.

Neon: A managed cloud PostgreSQL service used to host the database.

HTTP APIs and JSON: Allow the frontend and backend, and LogScope and LogHealth, to exchange structured information.

Vercel: Deployment platform used for the web applications, subject to runtime and deployment configuration.

The stack was selected to keep the application modular and relatively lightweight. Python is suitable for text processing, Flask provides a simple API layer, PostgreSQL offers structured persistent storage, and Neon simplifies hosted database management. Separating the frontend, backend, analysis modules, and database makes the application easier to maintain and extend.

6. Datasets Used (If Any)

The core workflow analyzes log files supplied by users rather than depending on a single fixed training dataset. Intended log categories include SQL and database logs, Minecraft server logs, Python application logs, and Java application logs, subject to the formats and diagnostic patterns recognized by the implementation.

Sample and synthetic logs can be used for development and testing. Examples may include missing database tables or columns, connection failures, deadlocks, Python exceptions, Java exceptions, and Minecraft server or plugin errors.

No machine-learning training dataset is required for the deterministic rule-based diagnosis approach described here. The reusable knowledge base is separate from a training dataset: it stores generalized diagnostic patterns and solutions for retrieval. A public dataset should be named only if it was actually used in the submitted implementation.

7. Prompts Used for Website Construction (If Applicable)

AI-assisted development tools may be used during implementation to help draft code, troubleshoot errors, refine the interface, and document the application. The following are representative prompt descriptions based on development tasks; they are not a verbatim record of every prompt used.

- Build a lightweight Flask web application that accepts uploaded log files and displays structured analysis results.
- Create modular Python components for parsing logs, detecting errors, classifying severity, grouping repeated patterns, and generating deterministic diagnostic explanations.
- Connect the Flask application to Neon-hosted PostgreSQL and keep database credentials in environment variables.
- Improve error matching so related log entries can be grouped and likely causes distinguished from secondary symptoms.
- Add reusable diagnostic knowledge that stores generalized, sanitized solutions separately from temporary uploaded logs.
- Add anonymous community fix submissions and individual upvotes, with safeguards against repeated voting and sensitive submissions.
- Refine the dark-themed results interface so errors are displayed compactly and detailed information is revealed when an error is selected.
- Integrate LogScope and LogHealth through backend APIs so errors can be sent for further analysis and analyzed logs can be imported into the other application.

These prompts describe development assistance and should not be treated as proof that every requested feature is fully implemented. The final application requires code review and testing.

8. API Integration

LogScope is designed to integrate with LogHealth through HTTP APIs. The purpose is to let the applications exchange diagnostic information and reduce the need for users to manually copy error details between them.

8.1 LogHealth to LogScope: Analyse Further

9. LogHealth displays an “Analyse further” action for an error.
10. LogHealth sends the relevant error details to a LogScope backend API.

11. LogScope processes the supplied information using its parser, diagnostic rules, and knowledge lookup.
12. LogScope returns a structured diagnosis response.
13. LogHealth displays the returned explanation and troubleshooting information in its interface.

8.2 LogScope to LogHealth: Import Log

14. After analyzing a log, the user selects “Import log to LogHealth”.
15. LogScope converts relevant analysis results into the JSON structure expected by LogHealth.
16. LogScope sends the structured data to a LogHealth API endpoint.
17. LogHealth validates and imports the received information.
18. The user is directed to LogHealth to view the imported information.

The integration uses configurable deployment URLs and server-side API credentials. API secrets should be stored as environment variables and must not be exposed in browser-side JavaScript or committed to source control. Request and response fields must match the API contracts implemented by both applications. Live cross-application operation should be tested after deployment.

9. Users, Inputs, and Outputs

9.1 Intended users

- Software developers investigating application exceptions and failures.
- Database administrators and developers working with SQL and database error logs.
- Minecraft server administrators investigating crashes, plugin errors, and runtime issues.
- IT support and troubleshooting teams reviewing application logs.
- Students learning programming and interpreting technical errors.
- Small software teams looking for a focused log diagnostic workflow.

9.2 Inputs

The primary input is a user-uploaded log file. Depending on the supported parser and supplied content, it may contain plain-text log entries, timestamps, severity labels, exception messages, stack traces, SQL error messages, server events, or application-specific output.

Intended categories include SQL/database, Minecraft, Python, and Java logs. The system can also receive individual error details from LogHealth through the API integration. For community contributions, a user may submit a problem description, the fix they applied, and whether they report that it worked.

9.3 Outputs

- A structured list of detected errors and warnings.
- Error categories and severity information where identifiable.
- Grouped repeated error patterns and analysis statistics.
- Likely causes and normal rule-based troubleshooting suggestions.
- Matching community fixes displayed separately when available.

- Individual community-fix votes and associated confidence information, according to the implemented scoring method.
- Knowledge-base statistics, such as stored rules and community contributions, where supported by the database.
- Structured JSON responses for API communication with LogHealth.

Diagnostic output is advisory. Users should inspect the evidence and validate a proposed fix in the relevant application environment before applying it.

10. References

19. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
20. PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/
21. Neon. Neon Documentation. https://neon.tech/docs
22. MDN Web Docs. HTML, CSS, and JavaScript Documentation. https://developer.mozilla.org/
23. Vercel. Vercel Documentation. https://vercel.com/docs
24. Elastic. Elastic Stack Documentation. https://www.elastic.co/guide/
25. Splunk. Splunk Documentation. https://docs.splunk.com/